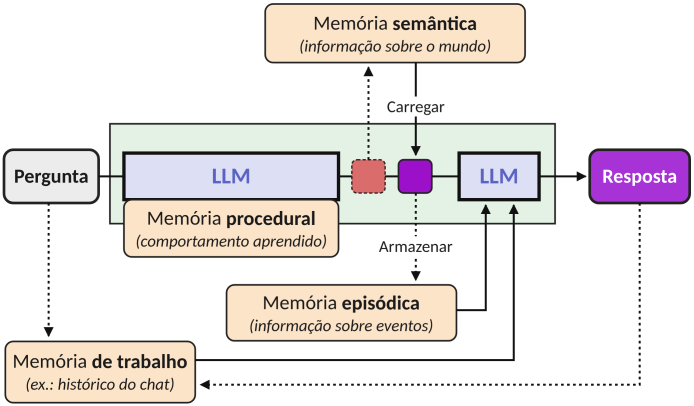
Unidade 6 — Memória em Agentes LLM

Aplicações de Modelos de Linguagem Natural

Por que agentes precisam de memória

- Sozinhos, os LLMs "esquecem": não se lembram de conversas passadas nem das ações que já tomaram.
- Sem memória:
  - um **assistente pessoal** não lembraria as conversas anteriores;
  - um **agente de programação** não entenderia o projeto inteiro;
  - um agente esqueceria que já executou uma ação e a **repetiria**.
- A memória inclui não só as ações passadas do agente, mas também **informação externa** (por exemplo, a documentação e as *issues* de um projeto, para um agente de programação).
- Memória envolve tanto **lembrar** quanto **armazenar** informação nova, e também decidir o que guardar, como guardar e o que esquecer.
- Com memória, o agente pode lembrar erros e experiências malsucedidas e se sair melhor em tarefas parecidas no futuro.
- Ferramentas e planejamento também dependem de memória: como usar uma ferramenta ou seguir um plano se o agente os esquece?

3.2 Tipos de memória



- a memória **procedural** está "colada" ao LLM, pois faz parte do seu comportamento;
- a memória **semântica** é carregada de uma fonte externa;
- a memória **episódica** armazena o que o agente fez;
- a memória **de trabalho** guarda a pergunta e a resposta da conversa em andamento.

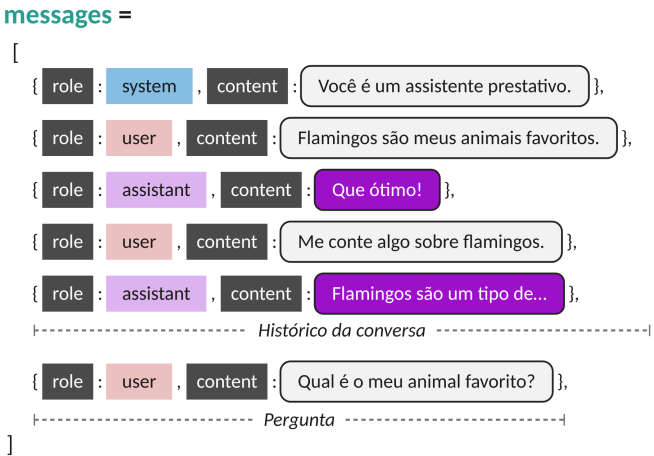
3.3 Memória de curto prazo: memória de conversa

- O histórico da conversa serve de contexto para gerar respostas. Ele é organizado como uma lista de **mensagens**, cada uma com um papel (*role*):
  - o **prompt de sistema** (`system`);
  - a pergunta do **usuário** (`user`);
  - a resposta do **assistente** (`assistant`).
- Essas mensagens são atualizadas a cada resposta e reenviadas ao LLM como entrada.

Em outras palavras

O LLM **não lembra** de fato as conversas, ele **é informado** sobre elas ao inseri-las no prompt.

3.3 Memória de curto prazo: o histórico de mensagens



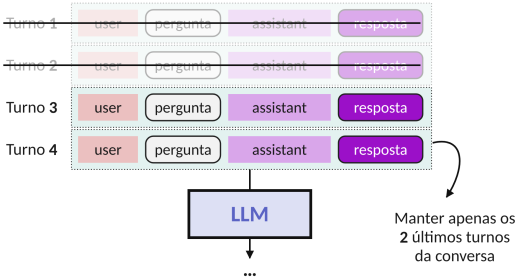
Memória de curto prazo: janela de contexto

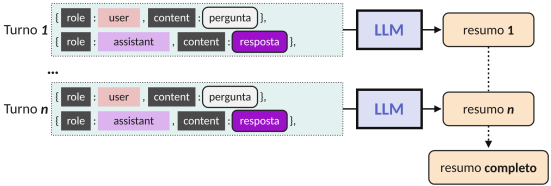
- A maioria dos LLMs tem uma **janela de contexto** (*context window*) limitada: o número máximo de tokens que consegue processar, contando entrada e saída juntas.
- Conforme o histórico cresce, cresce o número de tokens. Se ficar grande demais, a resposta pode ser cortada no meio, ou o LLM pode nem conseguir processar o prompt. Além disso, quanto mais informação no prompt, mais difícil é para o LLM prestar atenção em tudo.

Por isso, existem técnicas para fornecer o histórico sem lotar a janela:

- **Corte (*trimming*)**
- **Sumarização (*summarization*)**

3.3 Memória de curto prazo: corte e sumarização





Manter o histórico exige, portanto, entender o que importa: todo o histórico, só o recente ou um resumo? Para conversas curtas, o histórico completo funciona bem; para longas sequências de ações, talvez não.

Memória de longo prazo: RAG como memória

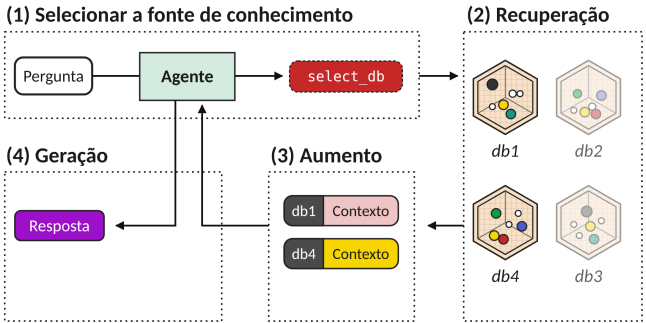
- À medida que crescem o histórico e o número de ações, cresce a necessidade de memória de longo prazo, que pode incluir também conhecimento externo ou proprietário.
- Ela costuma envolver **bancos de dados externos** consultáveis, com informações sobre execuções anteriores do agente (memória episódica) ou sobre o contexto da aplicação, como os documentos de uma organização (memória semântica).
- O método mais comum é o **RAG**, que já estudamos nas Unidades 4 e 5. Do ponto de vista dos agentes, **o banco de dados vetorial funciona como a memória de longo prazo do LLM**.

3.4 Memória de longo prazo: MemoryBank

- Um exemplo de memória mais elaborada é o **MemoryBank**, um mecanismo que armazena as experiências das conversas em um banco separado e o **atualiza continuamente**.
- Ele é inspirado na **curva do esquecimento de Ebbinghaus**: esquecemos rápido, mas a repetição espaçada reduz o esquecimento. Assim, as memórias recuperadas com frequência ficam mais tempo, e as que não são acessadas por muito tempo podem ser removidas.
- O MemoryBank usa três tipos de memória:
  1. o **histórico bruto**;
  2. **resumos de eventos passados**, gerados por um LLM;
  3. um **retrato do usuário** (traços de personalidade e emoções), sempre enviado como contexto adicional.

Memória de longo prazo: RAG agêntico

- No RAG simples, o LLM recebe o que foi recuperado, mas **não tem controle sobre o que é recuperado**.
- No **RAG agêntico** (*agentic RAG*), um agente acessa as bases externas **como ferramentas** e decide de qual fonte buscar, quantas vezes buscar e quando já tem contexto suficiente para responder.



3.4 Memória de longo prazo: A-MEM e Search-o1

A-MEM

- Memória agêntica inspirada no método de anotações **Zettelkasten**, em que cada nota guarda uma única unidade de conhecimento (atomicidade) e se liga a outras notas (notas hipertextuais).
- Cada nota de memória contém a interação original, o horário, palavras-chave, etiquetas e uma descrição contextual geradas pelo LLM. A nota é transformada em embedding e comparada às outras memórias, e o LLM decide a quais delas ligá-la. Em seguida, as memórias antigas ligadas também são atualizadas, criando uma memória que **evolui com o tempo**.

Search-o1

- Faz buscas **durante o raciocínio** do LLM, usando marcadores especiais para iniciar uma busca e indicar o seu resultado.
- Como os documentos recuperados podem ser longos e conter informação irrelevante, um módulo chamado **Reason-in-Documents** condensa a informação recuperada antes de inseri-la no raciocínio.

Engenharia de contexto: fontes de contexto

Além dos tipos de memória já vistos, há outras formas de contexto que podem ser dadas ao agente:

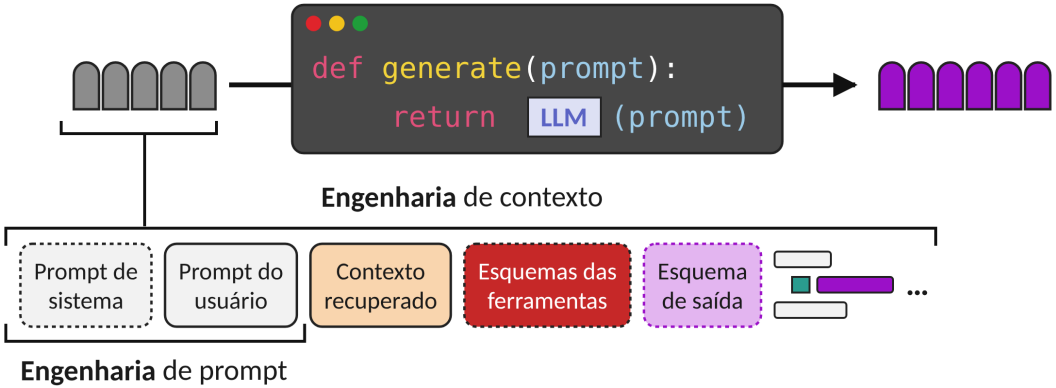
Fontes de contexto

Prompt de sistema → regras e contexto central do agente (memória procedural)
Histórico da conversa → conversa entre usuário e assistente e os "pensamentos" do LLM (memória de trabalho)
Experiências passadas → eventos, ações e observações de ferramentas, fatos sobre o usuário (memória episódica)
Informação recuperada → informação externa, normalmente em um banco vetorial acessado via RAG (memória semântica)

- Ou seja, **o prompt do usuário é só uma parte do contexto total do LLM**.
- A **engenharia de contexto** (*context engineering*) consiste em encontrar o melhor contexto, aquele que maximiza a qualidade da saída do LLM para uma tarefa.

3.5 Engenharia de contexto: prompt × contexto

Enquanto a **engenharia de prompt**, estudada nas Unidades 1 e 2, otimiza os prompts de sistema e do usuário, a **engenharia de contexto** otimiza o contexto inteiro.



3.5 Engenharia de contexto: por que não encher a janela de contexto?

As janelas de contexto cresceram muito (o **Gemini 1.5** chegou a **um milhão de tokens** em fevereiro de 2024), e poderia parecer que basta colocar tudo nelas, dispensando até o RAG. Há, porém, três argumentos contra essa ideia:

1. O teste da **agulha no palheiro** (*needle in a haystack*) coloca um fato aleatório no meio de um texto longo e pede ao modelo que o encontre. Mas encontrar a agulha é só uma tarefa de recuperação, e não mede a compreensão de contextos longos.
2. Benchmarks como o **RULER** mostraram que modelos que iam bem no teste da agulha tinham **quedas fortes de desempenho** com contextos maiores. Outros estudos chegaram à mesma conclusão, e alguns chamaram o fenômeno de **apodrecimento do contexto** (*context rot*).
3. **Custo e latência:** mais tokens significam respostas mais lentas e mais memória de GPU (VRAM). Despejar tudo no contexto é receita para o fracasso.

A regra da engenharia de contexto

Colocar **a informação certa, no lugar certo e no formato certo**: nem informação demais ou irrelevante (o custo sobe e o desempenho cai), nem de menos (o LLM age sem entender a situação). É, em grande parte, um problema de **arquitetura**: rastrear, armazenar e recuperar informação de forma eficiente.